Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251188
Nama Lengkap	ANGELA REVALINE SETIAWAN
Minggu ke / Materi	10 / Manajemen File

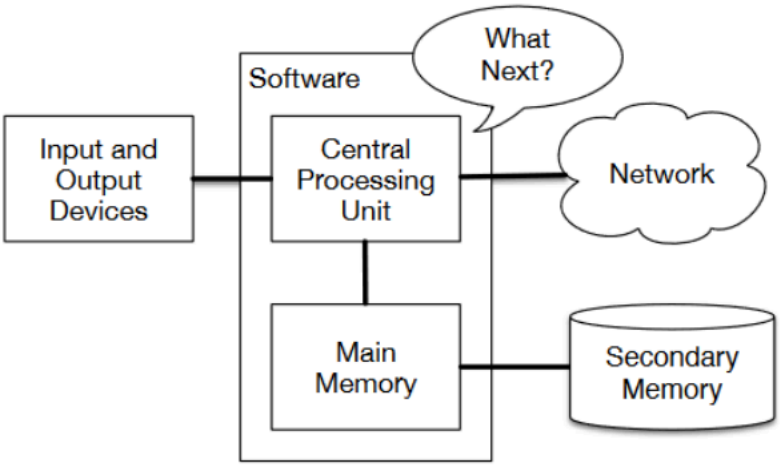
PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

1. Pengantar File

- Semua data yang ada dalam program disimpan di dalam memory
- Data yang disimpan di memori utama (RAM) bersifat sementara (volatile) yang berarti ketika program telah selesai dijalankan dan komputer dimatikan, maka data akan ikut hilang.
- Agar data dapat tersimpan secara permanen, maka digunakan **secondary memory** (hard disk, SSD, dll).

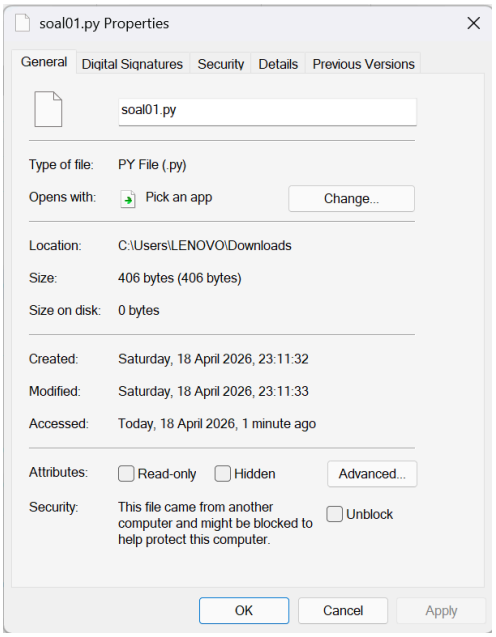


Gambar 10.1 : Secondary Memory di Komputer.

- Main Memory = RAM
- Secondary Memory = storage yang dibutuhkan

- File di simpan pada secondary memory sehingga file dapat digunakan untuk menyimpan data dari program dan tidak akan hilang walaupun komputer dimatikan.
- Fil adalah kumpulan bit data yang disimpan di secondary memory secara permanen.
- File bisa berupa :
    a) file sistem
    b) file program (binary)
    c) file teks
    d) file multimedia

- File memiliki properti seperti :
    a) nama file
    b) ukuran
    c) letak di hard disk
    d) owner
    e) hak akses
    f) tanggal akses



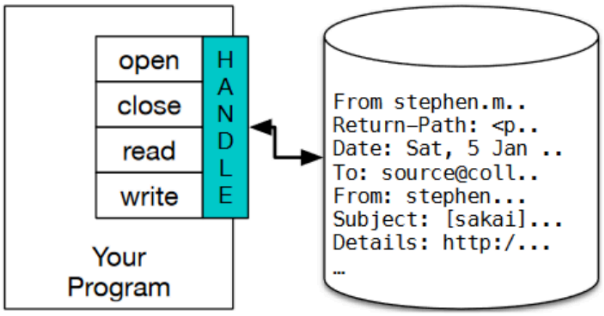
Gambar 10.2 : Contoh Property File

Secondary memory tidak hanya terbatas pada hard disk local, tetapi juga termasuk penyimpanan cloud seperti Google Drive atau Dropbox yang dapat diakses melalui API tertentu.

2. Pengaksesan File

Langkah-langkah yang harus dilakukan untuk dapat mengakses file :

1) Menyiapkan file dan path yang akan diakses.
2) Open file.
3) Lakukan sesuatu dengan file tersebut, seperti ditampilkan (read) isinya atau diubah / ditulisi (write).
4) Close file



Gambar 10.3 : Ilustrasi Handle File

Contoh program python :

```
handle = open('mbox-short-txt')
print(handle)
```

Output :

```
<_io.TextIOWrapper name='mbox-short.txt' mode='r' encoding='UTF-8'
```

Jadi, hasilnya berupa tampilan **nama file**, **modenya r = read**, dan **encoding** yang digunakan adalah **unicode UTF-8** dari sistem io pada Python.

Jika nama file tidak ada / tidak ditemukan, maka output akan error :

```
Traceback (most recent call last): File "main.py", line 1, in <module>
handle = open('tidak-ada.txt')
FileNotFoundError: [Errno 2] No such file or directory: 'tidak-ada.txt'
```

Contoh lain :

```
1 try:
2     with open('data.txt', 'r') as file:
3         print(file.read())
4 except FileNotFoundError:
5     print("File tidak ditemukan!")
```

* penggunaan **with** statement sangat disarankan dalam mengakses file karena secara otomatis akan menutup file setelah blok kode selesai.
* bahkan ketika terjadi error, file akan tertutup secara otomatis.
* penggunaan **with** juga bisa menangkap error spesifik seperti **FileNotFoundError** atau **PermissionError**.

Pada file text, biasanya file akan terdiri dari baris demi baris string. Cara pembacaan file teks biasanya juga menggunakan model baca baris demi baris untuk setiap string yang ditemukan sampai dengan EOF (End of File).

Contoh tampilan baris-baris dari sebuah file teks mbox-short.txt adalah pada gambar 10.4.

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
Date: Sat, 5 Jan 2008 09:12:18 -0500
To: source@collab.sakaiproject.org
From: stephen.marquard@uct.ac.za
Subject: [sakai] svn commit: r39772 - content/branches/
Details: http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772
```

Gambar 10.4 : Contoh file teks dan barisnya.

3. Manipulasi File

File teks biasanya dibaca baris per baris untuk menghindari pemborosan memori. Untuk bisa memanipulasi file maka harus dimulai dari membaca file terlebih dahulu.
Cara membaca file pada Python :

1) Siapkan file
2) Open file
3) Loop setiap baris pada file
4) Close file

```
handle = open('mbox-short-txt')
count = 0
for line in handle:
    count = count + 1
print('Line Count: ', count)
```

Output : Line Count: 1910

Gambar 10.5 : Cara pembacaan file

Cara menampilkan ukuran file teks dalam bytes, dapat menggunakan fungsi **len()** dari string yang ada pada file.

```
handle = open('mbox-short-txt')
hasil = handle.read()
print("Ukuran: " + len(hasil) = "bytes")
print("Huruf dari belakang sendiri mundur 16 huruf adalah: " + hasil[-16::1])
```

Gambar 10.6 : Penggunaan fungsi **len()** dalam menampilkan ukuran file teks dalam bytes.

Program di atas ini akan membuka file mbox-short.txt, menampilkan ukuran berapa banyak huruf yang ada pada file tersebut (kalau dianggap 1 karakter = 1 byte, maka bisa disebut juga ukuran berapa banyak karakter = ukuran file tersebut dalam byte), dan terakhir menampilkan string dari huruf paling belakang maju 16 huruf kedepan.
* jika ukuran file sangat besar maka lebih baik menggunakan teknik loop, daripada menggunakan read() karena perintah read() sangat boros memory.

Saat looping kita dapat menampilkan hanya kalimat yang diawali dengan “tanggal” saja, yaitu “Date :”.

```
handle = open('mbox-short-txt')
count = 1
for line in handle:
    if line.startswith("Date: ") and count <= 10:
        count += 1
        print(line)
```

Program di atas hanya akan menghasilkan output 10 baris yang berupa tanggal saja.

```
Date: Sat, 5 Jan 2008 09:12:18 -0500

Date: 2008-01-05 09:12:07 -0500 (Sat, 05 Jan 2008)

Date: Fri, 4 Jan 2008 18:08:57 -0500

Date: 2008-01-04 18:08:50 -0500 (Fri, 04 Jan 2008)

Date: Fri, 4 Jan 2008 16:09:02 -0500

Date: 2008-01-04 16:09:01 -0500 (Fri, 04 Jan 2008)

Date: Fri, 4 Jan 2008 15:44:40 -0500

Date: 2008-01-04 15:44:39 -0500 (Fri, 04 Jan 2008)

Date: Fri, 4 Jan 2008 15:01:38 -0500

Date: 2008-01-04 15:01:37 -0500 (Fri, 04 Jan 2008)
```

Di setiap baris ada baris kosong karena dari file mbox sudah terdapat newline di setiap baris dan ditambah perintah print yang kita buat, sehingga terjadi double newline. Dapat menggunakan rstrip pada line atau menggunakan perintah print tanpa newline.

4. Penyimpanan File

Dalam Python cara untuk menulis ke file adalah sama dengan cara membuka (open) file pada sub bab sebelumnya, hanya perlu mengubah metodenya saja yang tadinya r menjadi w seperti : **fout=open('output.txt','w')**. Lalu untuk menuliskan isi string ke dalam file output.txt langsung saja menggunakan perintah **while(<string>)** dan jangan lupa tutup file dengan **close()**.

```
1  handle = open('mbox-short-txt')
2  tulisan = "teks ini akan dituliskan ke file\n"
3  handle.write(tulisan)
4  handle.close()
```

Gambar 10.7 : Contoh program untuk menulis ke file.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

Source Code :

```
def perbedaan_kata(kalimat1, kalimat2):
    k1 = kalimat1.split()
    k2 = kalimat2.split()

    max_kata = max(len(k1), len(k2))
    beda = 0
    for i in range(max_kata):
        if i < len(k1):
            kata1 = k1[i]
        else:
            kata1 = "tidak ada"

        if i < len(k2):
            kata2 = k2[i]
        else:
            kata2 = "tidak ada"

        if kata1 != kata2:
            beda += 1
    return beda

def perbedaan_file():
    try:
        file1 = input("Masukkan nama file pertama: ")
```

```
        file2 = input("Masukkan nama file kedua: ")

        a = open(file1).readlines()
        b = open(file2).readlines()

        max_baris = max(len(a), len(b))

        print("Hasil Perbandingan:\n")

        for i in range(max_baris):
            if i < len(a):
                baris1 = a[i].strip()
            else:
                baris1 = "tidak ada"

            if i < len(b):
                baris2 = b[i].strip()
            else:
                baris2 = "tidak ada"

            if baris1 != baris2:
                print(f"Baris ke-{i+1} berbeda:")
                print(f"File 1 : {baris1}")
                print(f"File 2 : {baris2}")
                                print(f"Jumlah  kata  beda  :
{perbedaan_kata(baris1, baris2)}")

    except:
        print("File tidak ditemukan!")

perbedaan_file()
```

Penjelasan :

1. Buat fungsi **perbedaan_kata** dengan parameter kalimat1 dan kalimat2. (kalimat1 berisi teks dari file 1 dan kalimat2 berisi teks dari file 2)
2. Memecah kalimat berdasarkan spasi menjadi list menggunakan fungsi **split()** dan menyimpan dalam variabel k1 dan k2.
3. Membuat variabel **max_kata** untuk menyimpan jumlah kata maksimum dari k1 dan k2.
4. Mengecek kata di k1 dan k2 lalu membandingkannya.
5. Membuat fungsi **perbedaan_file()** untuk membandingkan file1 dan file2.
6. Menggunakan **try-except** untuk menanggani error jika file tidak ditemukan.
7. Menyimpan input dari user dalam variabel file1 dan file2.
8. Menggunakan fungsi **open** dan **readlines()** untuk membaca isi file1 dan file2.
9. Membuat variabel **max_baris** untuk menyimpan jumlah baris maksimum dari kedua file.
10. Mengecek perbedaan baris dalam file1 dan file2.
11. Lalu gunakan fungsi **print** untuk mengeluarkan output.

Output :

```
Masukkan nama file pertama: file01.txt
Masukkan nama file kedua: file02.txt
Hasil Perbandingan:

Baris ke-1 berbeda:
File 1 : Saya suka makan es krim
File 2 : Saya suka makan dimsum
Jumlah kata beda : 2
```

SOAL 2

Source Code :

```
nama_file = "file03.txt"
print(f"nama file1: {nama_file}")

file = open(nama_file)

for line in file:
    pisah = line.strip().split("||")
    soal = pisah[0].strip()
    jawaban = pisah[1].strip().lower()

    print(soal)
    jawaban_user = input("Jawab: ").lower()

    if jawaban_user == jawaban:
        print("Jawaban benar!\n")
    else:
        print("Jawaban salah!\n")

file.close()
```

Penjelasan :

1. Membuat variabel **nama_file** untuk menyimpan nama file.
2. Menggunakan fungsi **print** untuk menampilkan output nama file.
3. Menggunakan fungsi **open()** untuk membuka dan membaca file.
4. Untuk membaca file perbaris gunakan fungsi **perulangan(for)**.
5. Menghapus spasi dan memisahkan soal dan jawaban dengan fungsi **strip()** dan **split()**.
6. Indeks 0 adalah soal dan indeks 1 adalah jawaban.
7. Menyimpan input jawaban user dalam variabel **jawaban_user**.
8. Mengecek apakah jawaban user sama dengan jawaban yang ada.
9. Menggunakan **print** untuk mengeluarkan output.

Output :

```
nama file1: file03.txt
1 + 1 =
Jawab: 2
Jawaban benar!

Bendera Indonesia?
Jawab: merah putih
Jawaban benar!

Kota gudeg adalah:
Jawab: yogyakarta
Jawaban benar!

Komponen PC untuk penyimpanan file adalah...
Jawab: harddisk
Jawaban benar!

50 * 20 =
Jawab: 1000
Jawaban benar!
```